

ASSIGNMENT-11.2

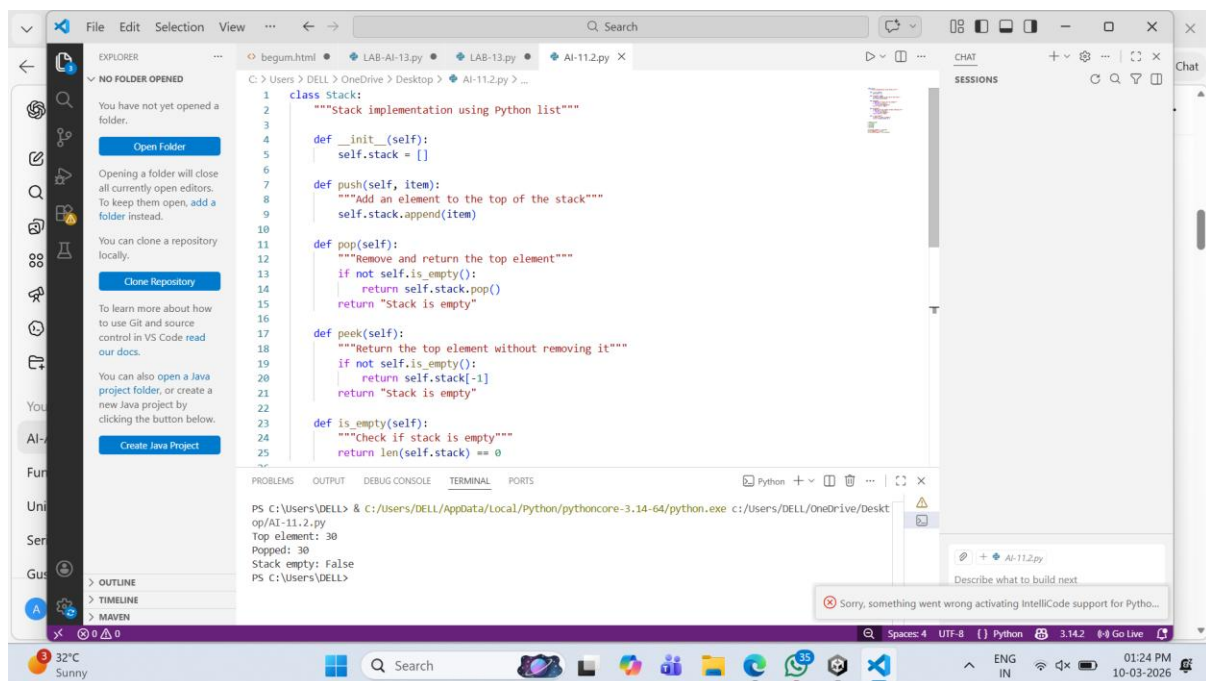
Name: YEROJU ANJALI

Hall Ticket: 2303A51416

Roll No: 29

Lab 11 – Data Structures with AI: Implementing Fundamental Structures

TASK-01: (Stack Using AI Guidance)



```
1 class Stack:
2     """Stack implementation using Python list"""
3
4     def __init__(self):
5         self.stack = []
6
7     def push(self, item):
8         """Add an element to the top of the stack"""
9         self.stack.append(item)
10
11    def pop(self):
12        """Remove and return the top element"""
13        if not self.is_empty():
14            return self.stack.pop()
15        return "Stack is empty"
16
17    def peek(self):
18        """Return the top element without removing it"""
19        if not self.is_empty():
20            return self.stack[-1]
21        return "Stack is empty"
22
23    def is_empty(self):
24        """Check if stack is empty"""
25        return len(self.stack) == 0
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\DELL> & C:\Users\DELL\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/DELL/OneDrive/Desktop/Al-11.2.py
op/Al-11.2.py
Top element: 30
Popped: 30
Stack empty: False
PS C:\Users\DELL>

Observation: STACK

The stack was implemented successfully using Python. The push, pop, peek, and empty-check operations worked correctly following the **LIFO** principle.

Task-02: (Queue Design)

The screenshot shows a Visual Studio Code editor with a Python file named `task-2.py`. The code implements a `Queue` class with methods `front`, `size`, and `is_empty`. An example usage is provided, enqueuing 5, 10, and 15, then printing the front element (5), dequeuing it, and printing the new front element (10) and queue size (2).

```
1 class Queue:
2     def __init__(self):
3         self.queue = []
4
5     def front(self):
6         if self.is_empty():
7             return "Queue is empty"
8
9     def size(self):
10        """Return queue size"""
11        return len(self.queue)
12
13    def is_empty(self):
14        return len(self.queue) == 0
15
16    # Example
17    q = Queue()
18    q.enqueue(5)
19    q.enqueue(10)
20    q.enqueue(15)
21
22    print("Front element:", q.front())
23    print("Dequeued:", q.dequeue())
24    print("Queue size:", q.size())
```

The terminal output shows the execution results:

```
PS C:\Users\DELL> & C:\Users\DELL\AppData\Local\Python\pythoncore-3.14-64\python.exe c:\Users\DELL\OneDrive\Desktop\AI-11-2.py
Top element: 30
Popped: 30
Stack empty: False
PS C:\Users\DELL> & C:\Users\DELL\AppData\Local\Python\pythoncore-3.14-64\python.exe c:\Users\DELL\OneDrive\Desktop\task-2.py
Front element: 5
Dequeued: 5
Queue size: 2
PS C:\Users\DELL>
```

Observation: QUEUE

The queue structure was created and tested. The enqueue and dequeue operations worked properly according to the **FIFO** principle.

TASK-03:(Singly Linked List Construction)

The screenshot shows a Visual Studio Code editor with a Python file named `task-3.py`. The code implements a `LinkedList` class with methods `insert` and `display`. An example usage is provided, inserting 30, 20, and 30, then displaying the list (30 -> 20 -> 30 -> None).

```
8 class LinkedList:
9     def insert(self, data):
10        new_node = Node(data)
11
12        if self.head is None:
13            self.head = new_node
14            return
15
16        temp = self.head
17        while temp.next:
18            temp = temp.next
19
20        temp.next = new_node
21
22    def display(self):
23        """Traverse and print list"""
24        temp = self.head
25        while temp:
26            print(temp.data, end=" -> ")
27            temp = temp.next
28        print("None")
```

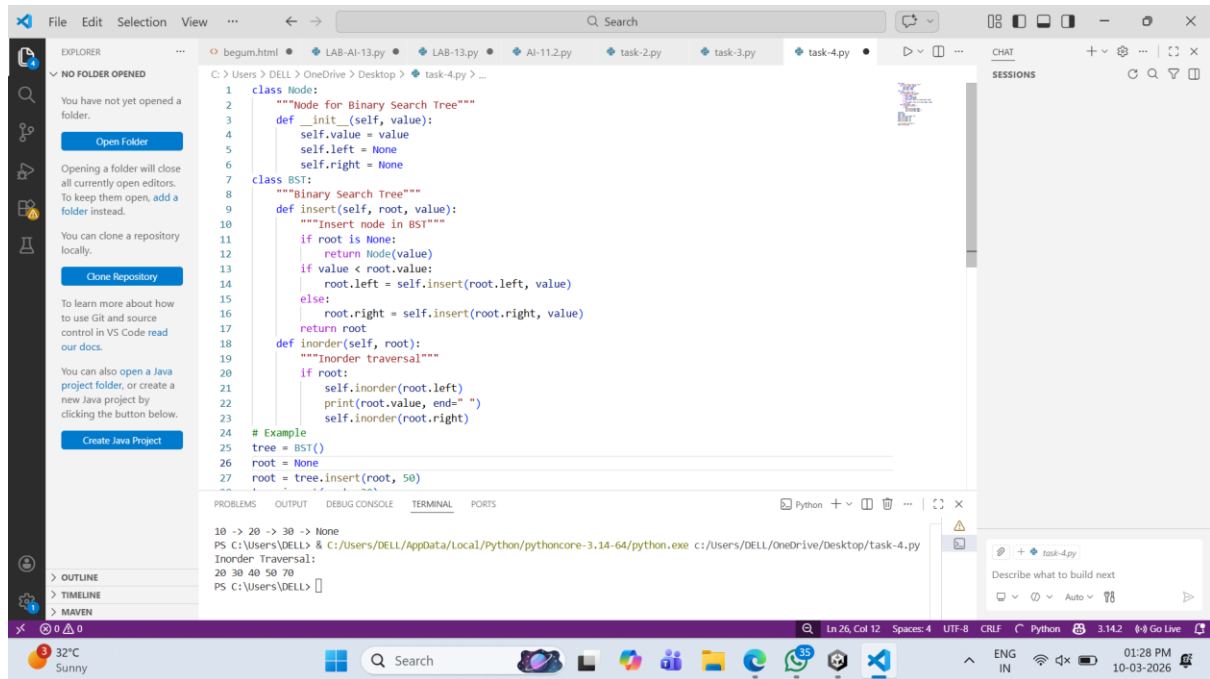
The terminal output shows the execution results:

```
Top element: 30
Popped: 30
Stack empty: False
PS C:\Users\DELL> & C:\Users\DELL\AppData\Local\Python\pythoncore-3.14-64\python.exe c:\Users\DELL\OneDrive\Desktop\task-2.py
Front element: 5
Dequeued: 5
Queue size: 2
PS C:\Users\DELL> & C:\Users\DELL\AppData\Local\Python\pythoncore-3.14-64\python.exe c:\Users\DELL\OneDrive\Desktop\task-3.py
Linked List:
10 -> 20 -> 30 -> None
PS C:\Users\DELL>
```

Observation:

A singly linked list was implemented with node insertion and traversal. The elements were stored dynamically and displayed correctly.

TASK-04: (Binary Search Tree Operations)



```
1 class Node:
2     """Node for Binary Search Tree"""
3     def __init__(self, value):
4         self.value = value
5         self.left = None
6         self.right = None
7
8 class BST:
9     """Binary Search Tree"""
10    def insert(self, root, value):
11        """Insert node in BST"""
12        if root is None:
13            return Node(value)
14        if value < root.value:
15            root.left = self.insert(root.left, value)
16        else:
17            root.right = self.insert(root.right, value)
18        return root
19
20    def inorder(self, root):
21        """Inorder traversal"""
22        if root:
23            self.inorder(root.left)
24            print(root.value, end=" ")
25            self.inorder(root.right)
26
27 # Example
28 tree = BST()
29 root = None
30 root = tree.insert(root, 50)
31
32 10 -> 20 -> 30 -> None
33 PS C:\Users\DELL> & C:\Users\DELL\AppData\Local\Python\pythoncore-3.14-64\python.exe c:\Users\DELL\OneDrive\Desktop\task-4.py
34 Inorder Traversal:
35 20 30 40 50 70
36 PS C:\Users\DELL>
```

Observation:

The BST was implemented with insertion and inorder traversal. The inorder traversal produced elements in **sorted order**.

TASK-05: (Hash Table Implementation)

```
1 class HashTable:
2     def __init__(self):
3         self.table = {}
4
5     def insert(self, key, value):
6         """Insert key-value pair"""
7         index = self.hash_function(key)
8         for i, pair in enumerate(self.table[index]):
9             if pair[0] == key:
10                 del self.table[index][i]
11                 return "Deleted"
12         self.table[index].append((key, value))
13         return "Key not found"
14
15     def search(self, key):
16         """Search for key"""
17         index = self.hash_function(key)
18         for pair in self.table[index]:
19             if pair[0] == key:
20                 return pair[1]
21         return "Key not found"
22
23     def delete(self, key):
24         """Delete key-value pair"""
25         index = self.hash_function(key)
26         for i, pair in enumerate(self.table[index]):
27             if pair[0] == key:
28                 del self.table[index][i]
29                 return "Deleted"
30         return "Key not found"
31
32     def hash_function(self, key):
33         return len(key) % len(self.table)
34
35 # Example
36 ht = HashTable()
37 ht.insert("apple", 10)
38 ht.insert("banana", 20)
39
40 print("Search apple:", ht.search("apple"))
41 print(ht.delete("apple"))
42 print("Search apple:", ht.search("apple"))
```

Terminal Output:

```
Inorder Traversal:
20 30 40 50 70
PS C:\Users\DELL> & C:/Users/DELL/AppData/Local/Python/pythoncore-3.14-64/python.exe c:/Users/DELL/OneDrive/Desktop/task-5.py
Search apple: 10
Deleted
Search apple: Key not found
PS C:\Users\DELL>
```

Observation:

A hash table was implemented using a hash function with collision handling. The insert, search, and delete operations worked correctly.